

@TheAutomationEngineer

Playwright Interview Questions & Answers

1. What is Playwright?

→ Answer:

Playwright is a next-generation test automation framework by Microsoft. It supports Chromium, WebKit, and Firefox, and enables reliable end-to-end testing for modern web apps.

2. What are the key features of Playwright?

→ Answer:

Auto-waiting, cross-browser support, parallel execution, tracing, screenshots, videos, network interception, device emulation, selectors (CSS, text, role, test id, etc.).

3. What languages does Playwright support?

→ Answer:

JavaScript/TypeScript, Python, .NET, and Java.

4. How is Playwright different from Selenium?

→ Answer:

Auto-waiting, no WebDriver, faster execution, reliable selectors, built-in tracing & videos.

5. What are the components of Playwright?

→ Answer:

Test Runner, Browser, BrowserContext, Page, Locator, Tracing, and Expect (assertions).

6. What is a BrowserContext in Playwright?

→ Answer:

It is an isolated browser session with its own cookies, storage, and preferences.

7. What is a Locator?

→ Answer:

Locator is a way to find elements in Playwright using user-friendly and reliable strategies.

8. How is auto-waiting implemented in Playwright?

→ Answer:

Playwright automatically waits for elements to be actionable before performing actions.

9. What is the role of Expect?

→ Answer:

Expect is used for assertions in Playwright. It has auto-retrying capability until the condition is met or timeout occurs.

10. What is the difference between Page and Context?

→ Answer:

Page represents a single tab or window. Context represents a browser session.

11. How do you handle a browser in Playwright?

→ Answer:

Using `browser.launch()` to start and `browser.close()` to close.

12. What is the use of Headless mode in Playwright?

→ Answer:

Headless mode runs tests without opening a UI. It is faster and useful in CI environments.

13. How do you capture screenshot in Playwright?

→ Answer:

Using `page.screenshot()` or `locator.screenshot()`.

14. How do you record video in Playwright?

→ Answer:

Configure `video` option in browser context to record test execution.

15. How do you handle multiple tabs in Playwright?

→ Answer:

Use `context.pages()` to get all pages and switch using `page.bringToFront()`.

16. How do you handle popups in Playwright?

→ Answer:

Use `page.waitForEvent('popup')` to capture and interact with the popup.

17. How do you handle frames in Playwright?

→ Answer:

Use `frameLocator()` or `contentFrame()` to interact with elements inside iframe.

18. How do you run tests in parallel?

→ Answer:

Use `parallel` in `playwright.config.*` or run multiple workers.

19. How do you configure test retries?

→ Answer:

Set retries in `playwright.config.*` to rerun failed tests automatically.

20. How do you capture API response in Playwright?

→ Answer:

Use `page.waitForResponse()` or `page.on('response')` to capture responses.

21. How do you mock APIs in Playwright?

→ Answer:

Use `page.route()` to intercept and mock API requests and responses.

22. How do you handle authentication?

→ Answer:

Use `storageState` to save and reuse authenticated state.

23. How do you integrate Playwright with CI/CD?

→ Answer:

Install dependencies and run tests using `npx playwright test` in CI pipelines.

24. How do you debug tests in Playwright?

→ Answer:

Use Playwright Inspector, headed mode, `PWDEBUG=1`, and console logs.

25. How do you generate test report in Playwright?

→ Answer:

Use `npx playwright show-report` to view the HTML report.

@The AutomationEngineer

Playwright Interview Questions & Answers

1.

What is Playwright and how does it differ from Selenium?

Answer:

Playwright is a modern end-to-end testing framework developed by Microsoft that supports Chromium, Firefox, and WebKit browsers.

Key **differences** from Selenium:

- **Auto-waits** for elements to be actionable before performing actions.
- **Built-in** support for modern web features (iframes, shadow DOM, web components).
- **Native** support for multiple browser contexts and pages.
- **Better handling** of dynamic content with built-in wait mechanisms.
- **Single API** for all browsers without needing browser-specific drivers.



2.

How do you handle multiple browser contexts in Playwright?

Answer:

```
import { test, chromium } from '@playwright/test';

test('multiple contexts example', async () => {
  const browser = await chromium.launch();

  // Create two independent browser contexts
  const context1 = await browser.newContext();
  const context2 = await browser.newContext();

  const page1 = await context1.newPage();
  const page2 = await context2.newPage();

  // Each context has independent cookies/storage
  await page1.goto('https://example.com');
  await page2.goto('https://example.com');

  await context1.close();
  await context2.close();
```

↓

```
  await browser.close();
});
```



⇒ @The AutomationEngineer ⇐

Playwright Interview Questions & Answers

3. Explain Playwright's auto-waiting mechanism

Answer:

Playwright automatically waits for elements to be actionable before performing actions. It checks:

- Element is **attached to DOM**
- Element is **visible**
- Element is **stable** (not animating)
- Element **receives events** (not obscured)
- Element is **enabled** (for interactive elements)

This eliminates most explicit waits and reduces flakiness in tests.



4. How do you handle iframes in Playwright?

Answer:

```
// Method 1: Using frameLocator
await page.frameLocator('iframe#myframe')
  .locator('button')
  .click();

// Method 2: Using frame() for named frames
const frame = await page.frame('frameName');
await frame.locator('button').click();

// Method 3: Using contentFrame() for iframe elements
const iFrameElement = page.locator('iframe#myframe');
const frame = await iFrameElement.contentFrame();
await frame.locator('button').click();
```



5. What are locator strategies in Playwright and which is recommended?

Answer:

Playwright supports multiple locator strategies:

- **page.getByRole()** - Recommended (accessibility-focused)
- **page.getByText()** - For visible text
- **page.getByLabel()** - For form fields with labels



≡ @TheAutomationEngineer ≡

Playwright Interview Questions & Answers

- `page.getByPlaceholder()` - For input placeholders
- `page.getByTestId()` - For data-testid attributes
- `page.locator()` - CSS/XPath selectors (last resort)

☆ Recommended priority: Role → Label → Placeholder → Text → TestId → CSS/XPath

6. How do you handle file uploads in Playwright?

Answer:

```
// Method 1: Using setInputFiles()
await page.locator('input[type="file"]').setInputFiles('path/to/file.pdf');

// Method 2: Multiple files
await page.locator('input[type="file"]').setInputFiles(['file1.pdf',
  'file2.pdf']);

// Method 3: Using file chooser event
const fileChooser = await Promise.all([
  page.waitForEvent('filechooser'),
  page.locator('button').click()
]);
await fileChooser.setFiles('path/to/file.pdf');

// Method 4: Buffer upload
await page.locator('input[type="file"]').setInputFiles({
  name: 'file.txt',
  mimeType: 'text/plain',
  buffer: Buffer.from('file content')
});
```

7. Explain Page Object Model (POM) implementation in Playwright

Answer:

```
pages/LoginPage.ts

export class LoginPage {
  constructor(private page: Page) {}

  private usernameInput = () => this.page.getByLabel('Username');
  private passwordInput = () => this.page.getByLabel('Password');
  private loginButton = () => this.page.getByRole('button', { name: 'Login' });

  async login(username: string, password: string) {
    await this.usernameInput().fill(username);
    await this.passwordInput().fill(password);
    await this.loginButton().click();
  }
}
```

≧ @TheAutomationEngineer ≦

Playwright Interview Questions & Answers

```
async goto() {  
  await this.page.goto('/login');  
}  
  
// test file  
test('login test', async ({ page }) => {  
  const loginPage = new LoginPage(page);  
  await loginPage.goto();  
  await loginPage.login('user@example.com', 'password123');  
});
```

8. How do you handle API testing in Playwright?

Answer:

```
import { test, expect } from '@playwright/test';  
  
test('API testing example', async ({ request }) => {  
  // GET request  
  const response = await request.get('https://api.example.com/users');  
  expect(response.ok()).toBeTruthy();  
  const users = await response.json();  
  
  // POST request  
  const createResponse = await request.post('https://api.example.com/users', {  
    data: {  
      name: 'John Doe',  
      email: 'john@example.com'  
    },  
    headers: {  
      'Authorization': 'Bearer token123'  
    }  
  });  
  expect(createResponse.status()).toBe(201);  
  
  // PUT/DELETE similar pattern  
});
```

9. How do you implement custom fixtures in Playwright?

Answer:

```
// fixtures.ts  
import { test as base } from '@playwright/test';  
import { LoginPage } from './pages/LoginPage';  
  
type MyFixtures = {  
  loginPage: LoginPage;  
  authenticatedPage: Page;  
};  
  
export const test = base.extend<MyFixtures>({  
  loginPage: async ({ page }, use) => {  
    const loginPage = new LoginPage(page);  
    await use(loginPage);  
  },  
  authenticatedPage: async ({ page }, use) => {  
    const loginPage = new LoginPage(page);  
    await loginPage.goto();  
    await loginPage.login('user@example.com',  
      'password123');  
    await use(page);  
  }  
});
```

Usage in tests:

```
import { test, expect }  
  from './fixtures';  
  
test('dashboard visible',  
  async ({ authenticatedPage }) => {  
    await authenticatedPage.goto('/dashboard');  
    await expect(authenticatedPage.getByRole('heading', { name: 'Dashboard' })).toBeVisible();  
  });
```

@The AutomationEngineer

Playwright Interview Questions & Answers

```
},
authenticatedPage: async ({ page }, use) => {
  await page.goto('/login');
  await page.fill('#username', 'testuser');
  await page.fill('#password', 'password');
  await page.click('button[type="submit"]');
  await use(page);
}
});
```

10. How do you handle network interception and mocking?

Answer:

```
// Intercept and mock API calls
test('mock API response', async ({ page }) => {
  await page.route('**/api/users', route => {
    route.fulfill({
      status: 200,
      contentType: 'application/json',
      body: JSON.stringify([
        { id: 1, name: 'Mock User' }
      ])
    });
  });
});

// Abort specific requests
await page.route('**/*.{png,jpg,jpeg}', route => route.abort());

// Continue with modifications
await page.route('**/api/**', route => {
  route.continue({
    headers: {
      ...route.request().headers(),
      'Authorization': 'Bearer mock-token'
    }
  });
});

await page.goto('/users');
```

What it does:

- ✓ Mocks API response for consistent testing
- ✓ Aborts static assets (e.g., images)
- ✓ Modifies request headers before continuing
- ✓ Helps in testing different scenarios without real backend

11. What are the different types of waits in Playwright?

Answer:

```
// 1. Auto-wait (built-in, most common)
await page.click('button'); // Automatically waits

// 2. Wait for element state
await page.locator('button').waitFor({ state: 'visible' });
await page.locator('button').waitFor({ state: 'hidden' });

// 3. Wait for navigation
await Promise.all([
  page.waitForURL('/dashboard'),
  page.click('a[href="/dashboard"]')
]);

// 4. Wait for load state
await page.waitForLoadState('networkidle'); // load, domcontentloaded, networkidle

// 5. Explicit wait (not recommended unless necessary)
await page.waitForTimeout(2000); // Waits for 2 seconds
```

Best Practice:

- ★ Prefer auto-wait and `waitFor*` methods.
- ★ Avoid hard waits (`waitForTimeout`).
- ★ Wait for conditions, not time.

≡ @The AutomationEngineer ≡

Playwright Interview Questions & Answers

```
page.waitForNavigation(),
page.click('a')
});

// 4. Wait for load state
await page.goto('https://example.com');
await page.waitForLoadState('networkidle');

// 5. Wait for selector
await page.waitForSelector('.dynamic-content');

// 6. Wait for function/condition
await page.waitForFunction(() => {
  return document.querySelectorAll('.items').length > 5;
});

// 7. Wait for timeout (not recommended)
await page.waitForTimeout(3000);

// 8. Wait for event
await page.waitForEvent('dialog');
```

Best Practices:

- ✓ Prefer auto-wait whenever possible.
- ✓ Use specific waits based on the scenario.
- ✓ Avoid hard waits (`waitForTimeout`).

12. How do you handle authentication and session storage?

Answer:

Method 1: Save and reuse authentication state

```
test('login once', async ({ page }) => {
  await page.goto('/login');
  await page.fill('#username', 'user');
  await page.fill('#password', 'pass');
  await page.click('button[type="submit"]');

  // Save storage state (cookies, localStorage, etc.)
  await page.context().storageState({ path: 'auth.json' });
});
```

playwright.config.ts

```
export default defineConfig({
  use: {
    storageState: 'auth.json' // Use saved state
  }
});
```

What gets saved in storageState?

- ★ Cookies
- ★ localStorage
- ★ sessionStorage
- ★ IndexedDB

Benefits:

- ✓ Login once, reuse across tests
- ✓ Faster test execution
- ✓ Consistent authenticated state

Method 2: Setup project for authentication

```
export default defineConfig({
  projects: [
    {
      name: 'setup',
      testMatch: /\.*\.\setup\.ts$/,
    },
    {
      name: 'chromium',
      use: {
        ...devices['Desktop Chrome'],
        storageState: 'auth.json'
      },
      dependencies: ['setup'] // Ensure setup runs first
    }
  ]
});
```

Execution Flow:

setup project
(Perform login & save state)

auth.json generated

All other projects use the
saved authenticated state

≡ @The AutomationEngineer ≡

Playwright Interview Questions & Answers

13. How do you perform visual regression testing?

Answer:

```
test('visual regression test', async ({ page }) => {
  await page.goto('https://example.com');

  // Full page screenshot comparison
  await expect(page).toHaveScreenshot('homepage.png');

  // Element screenshot comparison
  await expect(page.locator('.header'))
    .toHaveScreenshot('header.png');

  // With custom threshold
  await expect(page).toHaveScreenshot('page.png', {
    maxDiffPixels: 100,
    threshold: 0.2
  });

  // Masked areas (ignore dynamic content)
  await expect(page).toHaveScreenshot({
    mask: [page.locator('.timestamp')],
    fullPage: true
  });
});
```

Key Points:

- ✓ Compares current screenshots with baseline images.
- ✓ Detects unintended UI changes (visual bugs).
- ✓ Use masks to ignore dynamic content like timestamps, ads, etc.
- ✓ Adjust threshold or maxDiffPixels based on tolerance.

14. How do you handle browser contexts for parallel test isolation?

Answer:

```
import { test } from '@playwright/test';

// Each test gets a fresh browser context automatically
test('test 1', async ({ page, context }) => {
  // Isolated context with own cookies/storage
  await page.goto('https://example.com');
});

test('test 2', async ({ page, context }) => {
  // Different isolated context
  await page.goto('https://example.com');
});

// Custom context configuration
test.use({
  contextOptions: {
    viewport: { width: 1920, height: 1080 },
    geolocation: { latitude: 40.7128, longitude: -74.0060 },
    permissions: ['geolocation'],
    locale: 'en-US',
    timezoneId: 'America/New_York'
  }
});
```

Why use browser contexts?

- ✓ Each context has separate cookies, localStorage, sessionStorage, cache.
- ✓ Ensures test isolation for parallel execution.
- ✓ Prevents test data contamination.

Benefits:

- ☆ Reliable parallel testing
- ☆ Faster execution
- ☆ True test independence

Playwright Interview Questions & Answers

15. How do you debug Playwright tests?

Answer:

- ① **Headed mode**
`npx playwright test --headed`
- ② **Debug mode with inspector**
`npx playwright test --debug`
- ③ **Pause execution**

```
test('debug test', async ({ page }) => {
  await page.goto('https://example.com');
  await page.pause(); // Opens Playwright Inspector
});
```
- ④ **Slow motion**

```
test.use({
  launchOptions: { slowMo: 1000 } // 1000ms = 1s
});
```
- ⑤ **Screenshots on failure (in config)**

```
use: {
  screenshot: 'only-on-failure',
  video: 'retain-on-failure',
  trace: 'on-first-retry'
}
```
- ⑥ **Console logs**
`page.on('console', msg => console.log(msg.text()));`
- ⑦ **Playwright Inspector**
`PWDEBUG=1 npx playwright test`
- ⑧ **Trace viewer**
`npx playwright show-trace trace.zip`

Debugging Tips

- ✓ Use `--debug` or `page.pause()` to step through tests.
- ✓ Headed mode helps to see what's happening.
- ✓ Use `slowMo` to observe UI interactions.
- ✓ Traces give a timeline view of the test execution.
- ✓ Inspect network, console, and DOM in Inspector.

16. How do you handle dynamic content and shadow DOM?

Answer:

```
// Shadow DOM
test('shadow DOM handling', async ({ page }) => {
  // Method 1: Using piercing selector
  await page.locator('pierce=shadow-button').click();

  // Method 2: Navigate shadow root explicitly
  const shadowHost = page.locator('#shadow-host');
  const shadowRoot = await shadowHost.evaluateHandle(
    el => el.shadowRoot
  );
  await shadowRoot.locator('button').click();
});
```

Common Strategies for Dynamic Content

- Wait for element to appear: `await page.locator('text=loaded').waitFor();`
- Wait for element to be visible: `await expect(locator).toBeVisible();`
- Wait for network response: `await page.waitForResponse('**/api/data');`
- Wait for DOM to be stable: `await page.waitForLoadState('networkidle');`

Handling Dynamic Content

- ✓ Playwright auto-waits for elements to be actionable.
- ✓ Use locator-based strategies.
- ✓ Use `expect()` assertions to wait for conditions.
- ✓ Avoid fixed waits.

Shadow DOM Notes

- ✓ Use piercing selector for simple cases.
- ✓ For complex scenarios, navigate to `shadowRoot` and continue.
- ✓ Works with open shadow DOM only.
- ✓ Playwright handles shadow DOM better than Selenium.

@TheAutomationEngineer

Playwright Interview Questions & Answers

```
// Dynamic content
test('dynamic content', async ({ page }) => {
  // Auto-wait for element to appear
  await page.locator('.dynamic-element').click();

  // Wait for specific number of elements
  await page.locator('.list-item').nth(4).waitFor();

  // Wait for element to have text
  await expect(page.locator('.status'))
    .toHaveText('Complete', { timeout: 10000 });
});
```

Handling Dynamic Content

- ✓ Playwright auto-waits for elements.
- ✓ Use locator strategies with conditions.
- ✓ Use expect() for waiting until conditions are met.
- ✓ Avoid fixed waits.

17. How do you implement data-driven testing?

Answer:

```
// Method 1: Using test.describe with loop
const testData = [
  { username: 'user1', password: 'pass1' },
  { username: 'user2', password: 'pass2' },
  { username: 'user3', password: 'pass3' }
];

testData.forEach(data => {
  test('login with ${data.username}', async ({ page }) => {
    await page.goto('/login');
    await page.fill('#username', data.username);
    await page.fill('#password', data.password);
    await page.click('button[type="submit"]');
  });
});

// Method 2: Using fixtures from external file
import testCases from './testdata.json';
for (const testCase of testCases) {
  test('test case: ${testCase.name}', async ({ page }) => {
    // Test implementation using testCase data
  });
}

// Method 3: CSV/Excel data
import csv from 'csv-parser';
import fs from 'fs';
const users = [];
fs.createReadStream('users.csv')
  .pipe(csv())
  .on('data', (row) => users.push(row));
```

Why Data-Driven Testing?

- ★ Run the same test with multiple data sets.
- ★ Increase test coverage efficiently.
- ★ Easier to add or update test data.
- ★ Helps find edge cases.

Data Sources

- ✓ JSON files
- ✓ CSV / Excel files
- ✓ Databases
- ✓ APIs

18. How do you handle alerts, dialogs, and popups?

Answer:

1. Alerts

```
page.on('dialog', async dialog => {
  console.log(dialog.message());
  await dialog.accept(); // OK
  // await dialog.dismiss(); // Cancel
});
```

Usage:

```
page.on('dialog', async dialog => {
  if (dialog.type() === 'alert')
    await dialog.accept();
});
```

2. Confirm Dialogs

```
page.on('dialog', async dialog => {
  if (dialog.type() === 'confirm') {
    console.log(dialog.message());
    await dialog.accept(); // OK
    // await dialog.dismiss(); // Cancel
  }
});
```

Alternative:

```
const [dialog] = await Promise.all([
  page.waitForEvent('dialog'),
  page.click('button#confirm')
]);
await dialog.accept();
```

3. Prompt Dialogs

```
page.on('dialog', async dialog => {
  if (dialog.type() === 'prompt') {
    await dialog.accept('my text');
    // await dialog.dismiss();
  }
});
```

Example:

```
page.on('dialog', async dialog => {
  await dialog.accept('John Doe');
});
await page.click('button#open-prompt');
```



Best Practice:

- ✓ Always handle dialogs to avoid blocking tests.
- ✓ Use page.on('dialog', ...) early in the test.
- ✓ Combine with Promise.all() for reliable handling.

@The AutomationEngineer

Playwright Interview Questions & Answers

18. How do you handle alerts, dialogs, and popups?

Answer:

```
test('handle dialogs', async ({ page }) => {  
  // 1. Alert  
  page.on('dialog', async dialog => {  
    console.log(dialog.message());  
    await dialog.accept(); // or dialog.dismiss()  
  });  
  // 2. Confirm dialog  
  page.on('dialog', async dialog => {  
    expect(dialog.type()).toBe('confirm');  
    await dialog.accept(); // or dialog.dismiss()  
  });  
  // 3. Prompt dialog  
  page.on('dialog', async dialog => {  
    await dialog.accept('My input text');  
  });  
  // 4. New window/tab popup  
  const [popup] = await Promise.all([  
    page.waitForEvent('popup'),  
    page.click('a[target="_blank"]')  
  ]);  
  await popup.waitForLoadState();  
  await popup.locator('h1').textContent();  
});
```

Key Points:

- ✓ Use `page.on('dialog')` to handle alerts, confirms, and prompts.
- ✓ Always `accept()` or `dismiss()` to avoid test blocking.
- ✓ Use `Promise.all()` to handle new page/tab popups.
- ✓ Wait for popup to load and then interact.
- ✓ Validate popup content before closing if needed.

19. How do you implement retry logic and test stability?

Answer:

1. Built-in Retries (playwright.config.ts)

```
// playwright.config.ts  
export default defineConfig({  
  retries: process.env.CI ? 2 : 0,  
  
  use: {  
    actionTimeout: 10000,  
    navigationTimeout: 30000,  
  
    // Automatic retries for assertions  
    expect: {  
      timeout: 5000,  
      toHaveScreenshot: {  
        maxDiffPixels: 100  
      }  
    }  
  }  
});
```

2. Custom Retry Logic

```
// Custom retry logic  
test('with custom retry', async ({ page }) => {  
  let attempts = 0;  
  const maxAttempts = 3;  
  while (attempts < maxAttempts) {  
    try {  
      await page.goto('https://flaky-site.com');  
      await expect(page.locator('h1')).toBeVisible();  
      break; // Success, exit retry loop  
    } catch (error) {  
      attempts++;  
      if (attempts === maxAttempts) throw error;  
      console.log(`Retry attempt ${attempts} failed. Retrying...`);  
      await page.waitForTimeout(1000); // Wait before retry  
    }  
  }  
});
```

Ways to Improve Test Stability

- ★ Use auto-wait for elements.
- ★ Use locator-based strategies.
- ★ Avoid hard waits (`waitForTimeout`).
- ★ Use proper timeouts.
- ★ Implement retry for flaky scenarios.
- ★ Keep tests independent and isolated.

Best Practices

- ✓ Retry only for known flaky scenarios.
- ✓ Don't overuse retries (may hide real issues).
- ✓ Log retry attempts for easier debugging.
- ✓ Fix root cause of flakiness whenever possible.



Remember: Stable tests build confidence, flaky tests create noise.



> @TheAutomationEngineer <

Playwright Interview Questions & Answers

19. What are soft assertions in Playwright?

Answer:

```
import { test, expect } from '@playwright/test';

// Soft assertions (continue on failure)
test('soft assertions', async ({ page }) => {
  await page.goto('https://example.com');

  const softExpect = expect.soft; // Enable soft assertions

  softExpect(page.locator('.title')).toHaveText('Expected');
  softExpect(page.locator('.subtitle')).toBeVisible();
  softExpect(page.locator('.badge')).toHaveText('NEW');
});
```

Key Points:

- ✓ Tests continue even if some assertions fail.
- ✓ All failures are reported at the end.
- ✓ Great for validating multiple UI elements in one test run.
- ✓ Use `expect.soft()` or `expect.configure({ soft: true })`.



How it works?



20. How do you optimize Playwright test execution?

Answer:

```
// playwright.config.ts
export default defineConfig({
  // Run tests in parallel
  workers: process.env.CI ? 4 : undefined,

  // Shard tests across multiple machines
  shard: { total: 4, current: 1 },

  // Reuse browser context
  fullyParallel: true,

  use: {
    // Reduce trace overhead
    trace: 'on-first-retry',
    video: 'retain-on-failure',

    // Faster navigation
    waitUntil: 'domcontentloaded',
  },

  // Test timeout
  timeout: 30000,

  // Group related tests
  projects: [
    { name: 'chromium',
      use: { ...devices['Desktop Chrome'] }
    }
  ]
});

// Test optimization techniques
test.describe.configure({ mode: 'parallel' });

// Skip unnecessary tests
test.skip(({ browserName }) => browserName !== 'chromium');

// Use API calls instead of UI for setup
test.beforeEach(async ({ request }) => {
  await request.post('/api/setup', { data: { user: 'test' } });
});
```

Ways to Optimize

- ✓ Run tests in parallel (workers).
- ✓ Shard tests across multiple machines.
- ✓ Reuse browser context with `fullyParallel`.
- ✓ Reduce traces to `on-first-retry`.
- ✓ Use faster navigation with `waitUntil: 'domcontentloaded'`.
- ✓ Group tests using projects.
- ✓ Skip unnecessary tests.
- ✓ Use API requests for setup instead of UI interactions.

Benefits

- ☆ Faster test execution
- ☆ Lower resource usage
- ☆ Scalable in CI/CD
- ☆ More stable and reliable runs

≡ @The AutomationEngineer ≡

Playwright Interview Questions & Answers

21. What is Playwright Codegen and how do you use it?

Answer: Playwright Codegen is a test generator tool that records user interactions and generates test code automatically.

1. Basic codegen

```
npx playwright codegen https://example.com
```

2. Codegen with specific browser

```
npx playwright codegen --browser=firefox https://example.com
```

3. Codegen with custom viewport

```
npx playwright codegen --viewport-size=1280,720 https://example.com
```

4. Codegen with device emulation

```
npx playwright codegen --device="iPhone 12" https://example.com
```

5. Codegen with authentication (load saved state)

```
npx playwright codegen --load-storage=auth.json https://example.com
```

6. Save generated test to file

```
npx playwright codegen --output=tests/generated.spec.ts https://example.com
```

7. Codegen with specific language

```
npx playwright codegen --target=python https://example.com
```

Targets: javascript, python, java, csharp

8. Codegen with custom user agent

```
npx playwright codegen --user-agent="Custom Bot" https://example.com
```

9. Record in a specific locale

```
npx playwright codegen --lang=es-ES https://example.com
```

Key Features:

- ✓ Records clicks, fills, navigation
- ✓ Generates locators using best practices
- ✓ Shows live preview of generated code
- ✓ Supports assertion generation (click Assert button)
- ✓ Can resume recording on existing tests



22. Explain Playwright UI Mode and its features

Answer: Playwright UI Mode is an interactive GUI for running, debugging, and analyzing tests.

1. Launch UI Mode

```
npx playwright test --ui
```

2. UI Mode with specific tests

```
npx playwright test tests/login.spec.ts --ui
```

3. UI Mode with headed browser

```
npx playwright test --ui --headed
```

4. UI Mode with project

```
npx playwright test --ui --project=chromium
```

5. UI Mode with grep (run matching tests)

```
npx playwright test --ui --grep="login"
```

Playwright UI Mode Features:

- ▶ Run tests interactively (test by test or all).
- ⚙ Step through tests in headed mode.
- 📄 Inspect DOM, console, network, and traces.
- 👁 View test steps, logs, screenshots, and videos.
- 🔄 Retry failed tests and re-run specific tests.
- ⚙ Edit tests directly and re-run.
- ✓ Supports breakpoints and debugging.

- 💡 UI Mode is great for debugging flaky tests.
- 🔍 Use it to understand test failures quickly.
- 🔍 Combine with traces for deeper analysis.



≡ @TheAutomationEngineer ≡

Playwright Interview Questions & Answers

```
# UI Mode with specific project
npx playwright test --project=chromium --ui
```

Key Features:

-  **1. Watch Mode:** Auto-runs tests on file changes
-  **2. Time Travel:** Step through test execution with DOM snapshots
-  **3. Pick Locator:** Interactive locator picker tool
-  **4. Network Tab:** View all network requests
-  **5. Console Logs:** See browser console output
-  **6. Screenshots:** View screenshots at each step
-  **7. Trace Viewer:** Integrated trace viewing
-  **8. Filtering:** Filter tests by name, status, project
-  **9. Debugging:** Set breakpoints, step through code

Why UI Mode?

- ✓ Powerful interactive debugging experience
- ✓ Visual test execution with step-by-step insights
- ✓ Faster troubleshooting
- ✓ Great for development and exploration

```
// UI Mode vs Headed Mode:
// UI Mode benefits:
// - Interactive debugging
// - Visual test execution
// - Time travel through test steps
// - Better for development
```

```
// Headed Mode:
npx playwright test --headed // Just shows browser, less control
```



23. How do you configure NPM scripts for Playwright in package.json?

Answer:

```
{
  "name": "playwright-project",
  "version": "1.0.0",
  "scripts": {
    "test": "playwright test",
    "test:headed": "playwright test --headed",
    "test:ui": "playwright test --ui",
    "test:debug": "playwright test --debug",
    "test:chrome": "playwright test --project=chromium",
    "test:firefox": "playwright test --project=firefox",
    "test:webkit": "playwright test --project=webkit",
    "test:all-browsers": "playwright test --project=chromium --project=firefox --project=webkit",
    "test:smoke": "playwright test --grep @smoke",
    "test:regression": "playwright test --grep @regression",
    "test:api": "playwright test tests/api",
    "test:e2e": "playwright test tests/e2e",
    "test:parallel": "playwright test --workers=4",
    "test:serial": "playwright test --workers=1",
    "test:shard": "playwright test --shard=1/4",
    "test:retry": "playwright test --retries=2"
  }
}
```

Tips:

- ✓ Use scripts to quickly run tests in different modes and projects.
- ✓ Combine --grep with tags like @smoke, @regression, etc.
- ✓ Use parallel/shard for faster execution in CI/CD.

≡ @The AutomationEngineer ≡

Playwright Interview Questions & Answers

playwright.config.ts (Common Configuration)

```
{
  "test:reporter": "playwright test --reporter=html",
  "test:report": "playwright show-report",
  "test:trace": "playwright show-trace",

  "test:ci": "playwright test --reporter=blob",
  "test:local": "playwright test --headed --workers=1",

  "codegen": "playwright codegen",
  "codegen:auth": "playwright codegen --load-storage=auth.json",

  "install:browsers": "playwright install",
  "install:chrome": "playwright install chromium",
  "install:deps": "playwright install-deps",

  "test:update-snapshots": "playwright test --update-snapshots",
  "test:list": "playwright test --list",

  "test:specific": "playwright test tests/login.spec.ts",
  "test:grep": "playwright test --grep 'login'",
  "test:grep-invert": "playwright test --grep-invert 'slow'",

  "pretest": "npm run lint",
  "posttest": "npm run test:report"
},
"devDependencies": {
  "@playwright/test": "^1.48.0",
  "@types/node": "^20.10.0",
  "typescript": "^5.3.3"
}
```

What do these scripts do?

- ✓ Generate and view reports (HTML)
- ✓ View trace for debugging
- ✓ CI-optimized run (blob report)
- ✓ Run tests in headed mode locally
- ✓ Generate tests via codegen
- ✓ Manage browsers & system deps
- ✓ Update and list tests
- ✓ Run specific tests / grep by name
- ✓ Run pre & post tasks
- ✓ Pin dev dependency versions

Usage Examples:

```
npm test           # Run all tests
npm run test:headed # Run in headed mode
npm run test:ui     # Open UI mode
npm run test:smoke  # Run smoke tests only
npm run test:chrome # Run in Chrome only
npm run codegen     # Start codegen tool
```

💡 Tip:

Custom npm scripts help standardize execution and simplify team workflows.

24. What are the essential Playwright dependencies and their purposes?

Answer:

devDependencies in package.json

```
{
  "devDependencies": {
    // Core Playwright package - required
    "@playwright/test": "^1.48.0",

    // TypeScript support (if using TS)
    "typescript": "^5.3.3",
    "@types/node": "^20.10.0",

    // Additional reporters
    "playwright-html-reporter": "^1.0.0",
    "allure-playwright": "^2.15.0",

    // Test utilities
    "dotenv": "^16.3.1" // Environment variables
  }
}
```

Dependency Purposes

- ★ @playwright/test → Core testing library (fixtures, assertions, runners)
- ★ typescript → Type safety & better development experience
- ★ @types/node → Node.js type definitions for TypeScript
- ★ playwright-html-reporter → Beautiful HTML test reports
- ★ allure-playwright → Allure test reporting integration
- ★ dotenv → Manage environment variables securely

✓ Best Practices

- ✓ Keep dependencies updated regularly.
- ✓ Use exact versions in CI to avoid unexpected breakages.
- ✓ Add only necessary packages to keep project lightweight.
- ✓ Document versions and setup in README.

@The AutomationEngineer

Playwright Interview Questions & Answers

```
"faker": "^6.6.6", // Test data generation

// API testing helpers
"axios": "^1.6.0",

// Excel/CSV data handling
"xlsx": "^0.18.5",
"csv-parser": "^3.0.0",

// Visual regression
"@playwright/test": "^1.48.0" // Built-in screenshot comparison
},
"dependencies": {
  // Usually keep empty - tests shouldn't have runtime dependencies
}
```

What these are used for?

- ✓ faker → Generate fake test data
- ✓ axios → API testing
- ✓ xlsx → Read/Write Excel files
- ✓ csv-parser → Read CSV files
- ✓ @playwright/test → Core test runner with built-in features (assertions, fixtures, screenshots, etc.)
- ✓ dependencies → Keep minimal for clean and fast tests

Key Dependencies Explained:

```
// @playwright/test - Core test runner
import { test, expect } from '@playwright/test';

// @types/node - Node.js type definitions
import * as fs from 'fs';
import * as path from 'path';

// dotenv - Environment configuration
import 'dotenv/config';
const API_URL = process.env.API_URL;

// faker - Test data generation
import { faker } from '@faker-js/faker';
const email = faker.internet.email();
```

Why these dependencies?

- ★ Type safety and IntelliSense with TypeScript
- ★ Easier file handling (fs, path)
- ★ Environment variables management
- ★ Realistic test data generation
- ★ Better maintainability and readability

Installation Commands:

```
# Install Playwright
npm install -D @playwright/test

# Install browsers
npx playwright install

# Install system dependencies
npx playwright install-deps

# Install specific browser
npx playwright install chromium

# Install with TypeScript
npm install -D @playwright/test typescript @types/node
```

Notes:

- ✓ Use -D to install dev dependencies.
- ✓ playwright install downloads browsers (Chromium, Firefox, WebKit).
- ✓ install-deps installs OS-level dependencies.
- ✓ Install specific browser to save time and space.

25. What are the most important Playwright terminal commands?

Answer:

Test Execution Commands:

# Run all tests	<code>npx playwright test</code>	Runs all tests in the project
# Run specific test file	<code>npx playwright test tests/login.spec.ts</code>	Runs a specific test file
# Run tests with UI mode	<code>npx playwright test --ui</code>	Opens Playwright Test UI
# Run tests in headed mode	<code>npx playwright test --headed</code>	Runs tests in headed browser
# Run tests in debug mode	<code>npx playwright test --debug</code>	Runs tests in debug mode (PW Inspector on failure)
# Run tests in a specific project	<code>npx playwright test --project=chromium</code>	Runs tests in a specific browser project

Utility Commands:

```
# Show help
npx playwright --help
Shows all available commands

# Show version
npx playwright --version
Shows installed Playwright version

# List projects
npx playwright test --list
Lists all configured projects

# Generate code (record test)
npx playwright codegen
Opens Codegen to record tests
```



Pro Tips:

- ✓ Use `--grep "name"` to run tests matching a pattern.
- ✓ Use `--reporter=html` to generate HTML report.
- ✓ Use `--workers=N` to run tests in parallel (faster execution).

≡ @The AutomationEngineer ≡

Playwright Interview Questions & Answers

26. How do you configure different environments using NPM scripts?

Answer:

1 package.json

```
{
  "scripts": {
    "test:dev": "BASE_URL=https://dev.example.com playwright test",
    "test:staging": "BASE_URL=https://staging.example.com playwright test",
    "test:prod": "BASE_URL=https://example.com playwright test",

    "test:dev:ui": "BASE_URL=https://dev.example.com playwright test --ui",
    "test:staging:smoke": "BASE_URL=https://staging.example.com playwright test --grep @smoke",

    "test:ci": "playwright test --reporter=blob,html",
    "test:ci:chrome": "playwright test --project=chromium --reporter=blob",

    "test:mobile": "playwright test --config=playwright.mobile.config.ts",
    "test:desktop": "playwright test --config=playwright.desktop.config.ts",

    "test:parallel:dev": "BASE_URL=https://dev.example.com playwright test --workers=4",
    "test:serial:prod": "BASE_URL=https://example.com playwright test --workers=1"
  }
}
```

What's happening?

- ✓ Each script sets `BASE_URL` for the environment.
- ✓ Additional options like `--ui`, `--grep`, `--project`, `--workers` help customize execution.
- ✓ Different configs for mobile and desktop.
- ✓ CI scripts generate reports suitable for pipelines.

2 playwright.config.ts

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  use: {
    baseURL: process.env.BASE_URL || 'http://localhost:3000',
    extraHTTPHeaders: {
      'Authorization': process.env.AUTH_TOKEN || ''
    }
  }
});
```

How it works

- ★ `BASE_URL` is read from environment variable.
- ★ Falls back to localhost if not set.
- ★ `AUTH_TOKEN` is used for authenticated requests.

3 .env files (examples)

```
# .env.dev
BASE_URL=https://dev.example.com
API_KEY=dev-key-123

# .env.staging
BASE_URL=https://staging.example.com
API_KEY=staging-key-456

# Usage with dotenv
npm run test:dev
npm run test:staging
```

Best Practices

- ✓ Store environment specific values in `.env` files (do not commit sensitive data).
- ✓ Use `dotenv` to load variables.
- ✓ Keep scripts simple and reusable.
- ✓ Use different configs for different devices or browsers.



Using NPM scripts + environment variables makes it easy to manage and switch between environments like dev, staging, and prod.

@The AutomationEngineer

Playwright Interview Questions & Answers

27. How do you use Playwright CLI for test filtering and execution control?

Answer:

```
# Filter by test title
npx playwright test -g "login"

# Multiple grep patterns
npx playwright test --grep "login|signup"

# Exclude tests
npx playwright test --grep-invert "slow"
npx playwright test -gv "slow"

# Combine grep and invert
npx playwright test --grep "@smoke" --grep-invert "@skip"

# Filter by file path
npx playwright test tests/auth/
npx playwright test tests/auth/login.spec.ts

# Filter by project
npx playwright test --project=chromium --project=firefox

# Global timeout
npx playwright test --global-timeout=3600000

# Test timeout
npx playwright test --timeout=30000

# Repeat tests
npx playwright test --repeat-each=3

# Forbid only (CI mode)
npx playwright test --forbid-ealy

# Pass with no tests
npx playwright test --pass-with-no-tests

# Ignore snapshots
npx playwright test --ignore-snapshots
```

Option	What it does
✓ -g, --grep	Run tests that match the title/description.
✓ --grep-invert, -gv	Exclude tests that match the pattern.
✓ File path	Run tests in specific file or folder.
✓ --project	Run tests in one or more projects/browsers.
✓ --global-timeout	Set max time for entire test run.
✓ --timeout	Set max time per test.
✓ --repeat-each	Repeat each test N times.
✓ --forbid-only	Fail run if .only is used in tests.
✓ --pass-with-no-tests	Exit 0 even if no tests are found.
✓ --ignore-snapshots	Ignore snapshot comparisons.

28. How do you configure and use Playwright reporters via terminal?

Answer:

Single Reporter

```
npx playwright test --reporter=list
npx playwright test --reporter=line
npx playwright test --reporter=dot
npx playwright test --reporter=html
npx playwright test --reporter=json
npx playwright test --reporter=junit
```

Multiple Reporters

```
npx playwright test \
  --reporter=list,html,junit
```

You can also configure in playwright.config.ts:

```
export default defineConfig({
  reporter: [
    ['list'],
    ['html', { open: 'never' }],
    ['junit', { outputFile: 'results.xml' }]
  ]
});
```

Common Built-in Reporters

Reporter	Description	Best for
list	Detailed list of tests with status	CI logs, quick overview
line	One line per test	Compact output in CI
dot	Progress dots	Long running suites
html	Rich HTML report	Local report with details
json	JSON output	Custom processing/integration
junit	JUnit XML report	CI tools (Jenkins, GitLab, etc.)

Tips

- ✓ Use html reporter for detailed investigation.
- ✓ Use junit for CI integration and test results publishing.
- ✓ Combine reporters to get both console summary and rich report.
- ✓ Customize output location in playwright.config.ts.